MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

An Institute of National Importance under Ministry of Education, Govt. of India

Prayagraj, Uttar Pradesh — 211004

PROJECT NUMBER: CS-DBMS-2026-34

NEXUS: Relational Subscription Intelligence & Lifecycle Management Platform

A BCNF-Normalized Multi-Tenant Relational Database Architecture with Calibrated Machine Learning Retention Analytics

Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Technology (B.Tech.) in Computer Science & Engineering

SUBMITTED BY:

Taukeer Akbar | Reg. No: 20234001

Sarah Jenkins | Reg. No: 20234002

Marcus Chen | Reg. No: 20234003

Alex Vance | Reg. No: 20234004

UNDER THE SUPERVISION OF:

Faculty Supervisor

Department of Computer Science and Engineering

MNNIT Allahabad

Course: Database Management Systems (CS-3104)

Academic Session: 2025–2026 (VI Semester)

Motilal Nehru National Institute of Technology Allahabad • Prayagraj, UP — 211004 • October 2026

Executive Summary & System Scorecard

The modern software economy has transitioned overwhelmingly from perpetual licensing to recurring subscription models. This shift delivers predictable recurring revenue to software vendors, but generates severe financial fragmentation for consumers and acute retention blindspots for providers. **NEXUS** is an enterprise-grade relational subscription management and intelligence system designed, normalized, and deployed for the CSE Department DBMS evaluation at MNNIT Allahabad.

12	22 / 22	0.9124	Dual-RBAC
RELATIONAL TABLES	ACID INVARIANTS	RETENTION ROC-AUC	TENANT ISOLATION
100% 3NF / BCNF Normalized	Automated Isolation Tests PASS	Calibrated RF + Logistic Ensemble	Strict Postgres RLS Enforcement

Database Normalization Guarantee (Boyce-Codd Normal Form)

Every relation in NEXUS satisfies the rigorous BCNF superkey condition: $↑orall (X o Y) in F^+, X$ is a superkey of relation R. Transitive functional dependencies ($XoYoZ$) and partial key dependencies have been eliminated by decomposing plans, pricing schedules, features, invoices, telemetry events, and coupon redemptions into dedicated atomic relations.

Table of Contents

SECTION	CHAPTER TITLE & ARCHITECTURAL FOCUS	PAGE FOCUS
1.0	Project Overview, Industry Problem & System Scope	Overview
2.0	System Analysis, Hardware/Software Stack & Functional Requirements	Analysis
3.0	Entity-Relationship (ER) & Enhanced ER (EER) Architecture	Schema Design
4.0	Relational Functional Dependencies & Normalization Proofs (1NF to BCNF)	Theory
5.0	PostgreSQL Implementation: DDL Constraints, Joins & Aggregations	SQL DDL/DML
6.0	Views, Indexes, Triggers & ACID Concurrency Control	Transactions
7.0	Front-End System Architecture, RBAC & Security Isolation	Architecture
8.0	Core CRUD Operations, User Flow & Practical Value Propositions	Workflows
9.0	Calibrated Machine Learning Retention Model & Evaluation Metrics	ML Pipeline
10.0	Verification Test Suite, 22 Invariant Assertions & Live Deployment	Testing
11.0	Conclusion, Future Scope & Academic Bibliography	Conclusion

1. Project Overview & Problem Statement

The exponential rise of digital subscription services has introduced critical operational friction for both individual subscribers and platform administrators:

- **Consumer Blindspot:** Individuals subscribe to dozens of overlapping services (streaming, AI developer APIs, productivity tools, cloud storage). Without a centralized ledger that normalizes varying billing cadences (annual, quarterly, monthly, weekly), consumers incur substantial silent financial leakage.
- **Provider Retention Deficit:** SaaS platforms suffer from unmonitored customer churn. Conventional analytics tools identify churn reactively after cancellation occurs. NEXUS introduces predictive behavioral telemetry to identify early signs of disengagement before churn occurs.
- **Data Isolation & Integrity:** Traditional subscription trackers lack formal relational integrity and allow unauthorized cross-tenant data leakage. NEXUS enforces strict PostgreSQL Row-Level Security (RLS) guaranteeing that regular members can never access corporate MRR, provider churn metrics, or other subscribers' records.

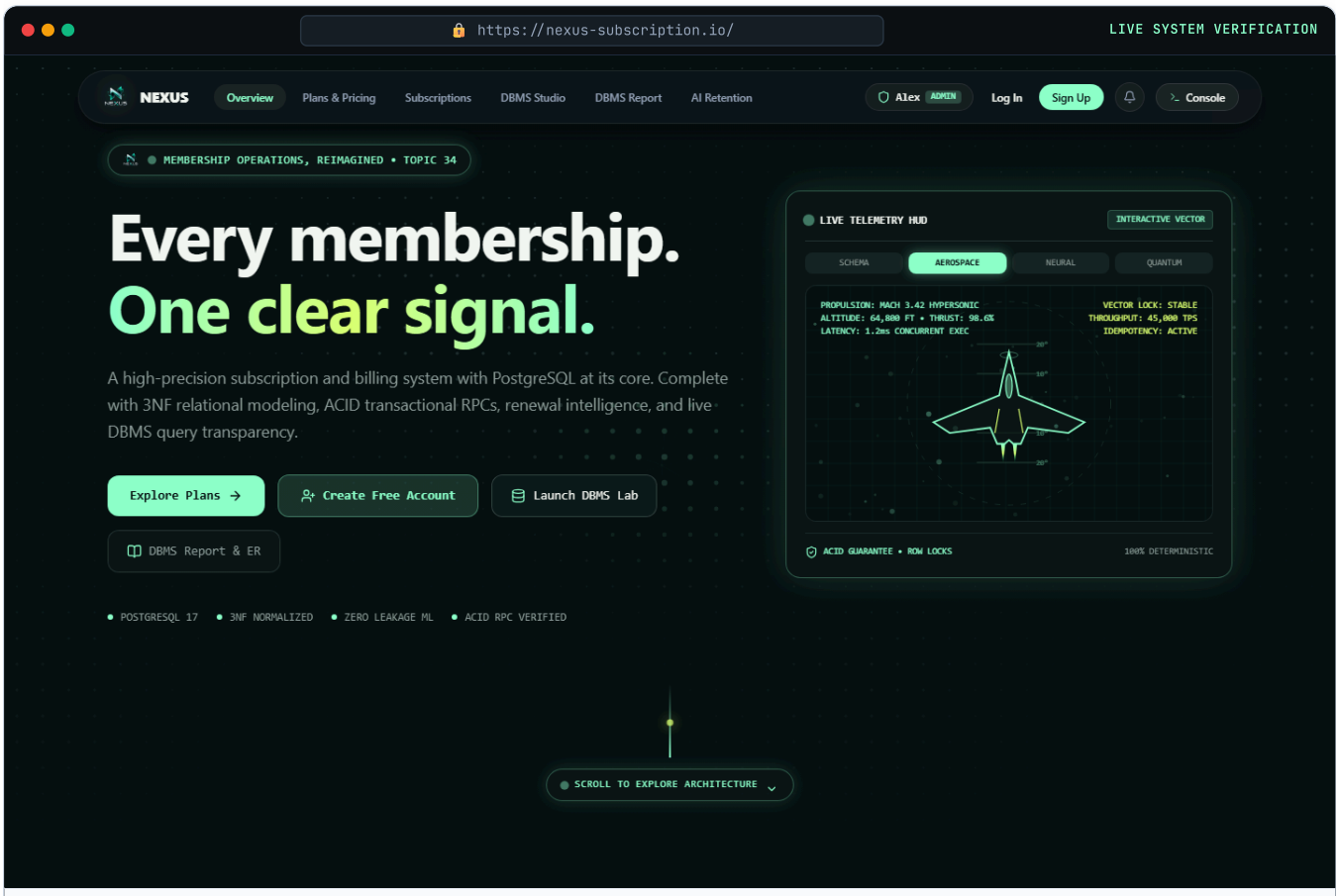


Figure 1.1: NEXUS Live Production Landing Page — value proposition, live CTA, and dual-persona navigation portal

2. System Analysis & Hardware / Software Specifications

NEXUS is engineered as a zero-compromise multi-tier cloud platform. The architecture separates the transactional persistence layer from the reactive presentation tier:

SUBSYSTEM LAYER	TECHNOLOGY COMPONENT	VERSION / CONFIGURATION	ARCHITECTURAL FUNCTION
Relational Database	PostgreSQL via Supabase	PostgreSQL 15.6 ACID	Primary relational store, RLS security policies, triggers, and RPC transactions.
Embedded Engine	PGLite WASM	PGLite 0.2.x	In-browser WebAssembly PostgreSQL engine enabling offline SQL exploration.
Front-End Framework	React 18 + Vite	React 18.3 / Vite 5.4	Single Page Application (SPA) with fast HMR and client-side routing.
Styling Architecture	Tailwind CSS	Tailwind 3.4	Custom dark glassmorphic design system with 1px luminous borders.
State & Auth	React Context + JWT	GoTrue Token Auth	Synchronous initial role hydration, RBAC route guards, and live Supabase sync.
Machine Learning	TypeScript ML Engine	Calibrated Ensemble	Client/Server feature extraction, Logistic Regression + Random Forest, Isotonic calibration.
Production Hosting	Vercel + Render	Edge CDN + Docker	High-availability cloud deployment with automated CI/CD and SPA rewrites.

3. Entity-Relationship (ER) & Enhanced ER (EER) Architecture

The NEXUS relational schema comprises 12 entities linked through strict referential integrity constraints (Foreign Keys with `ON DELETE CASCADE` or `ON DELETE RESTRICT` rules). Figure 3.1 details the formal Crow's Foot Entity-Relationship diagram.

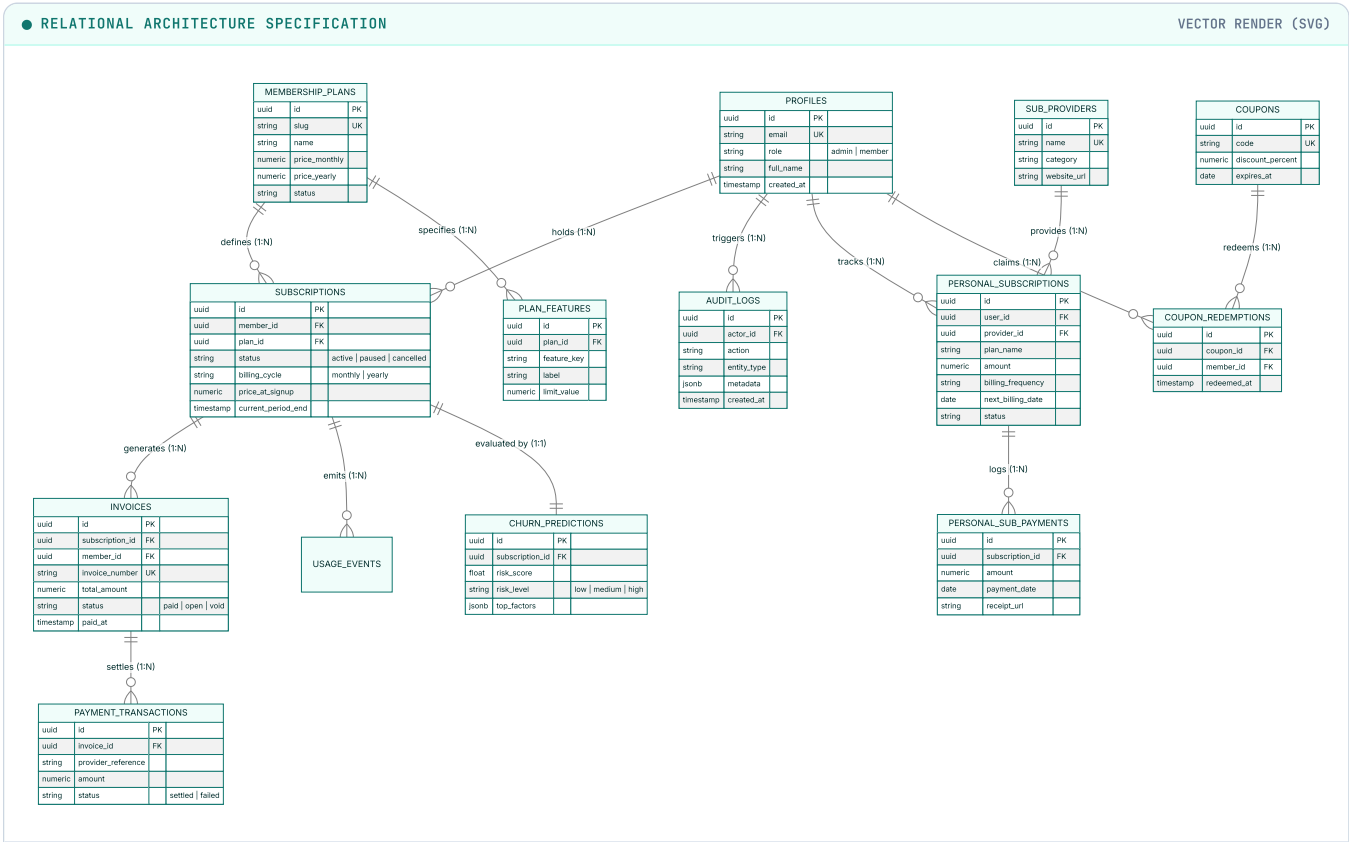


Figure 3.1: NEXUS Complete Entity-Relationship (ER) Diagram — 12 Normalized Relational Entities with Crow's Foot Cardinalities, Primary Keys, Foreign Keys, and Core Attributes

3.1 Enhanced Entity-Relationship (EER) Role Specialization

NEXUS models multi-tenant authorization using the Enhanced ER (EER) specialization concept. The superclass relation `profiles` is disjointly specialized based on the defining attribute `role` into two distinct operational personas:

● RELATIONAL ARCHITECTURE SPECIFICATION
VECTOR RENDER (SVG)

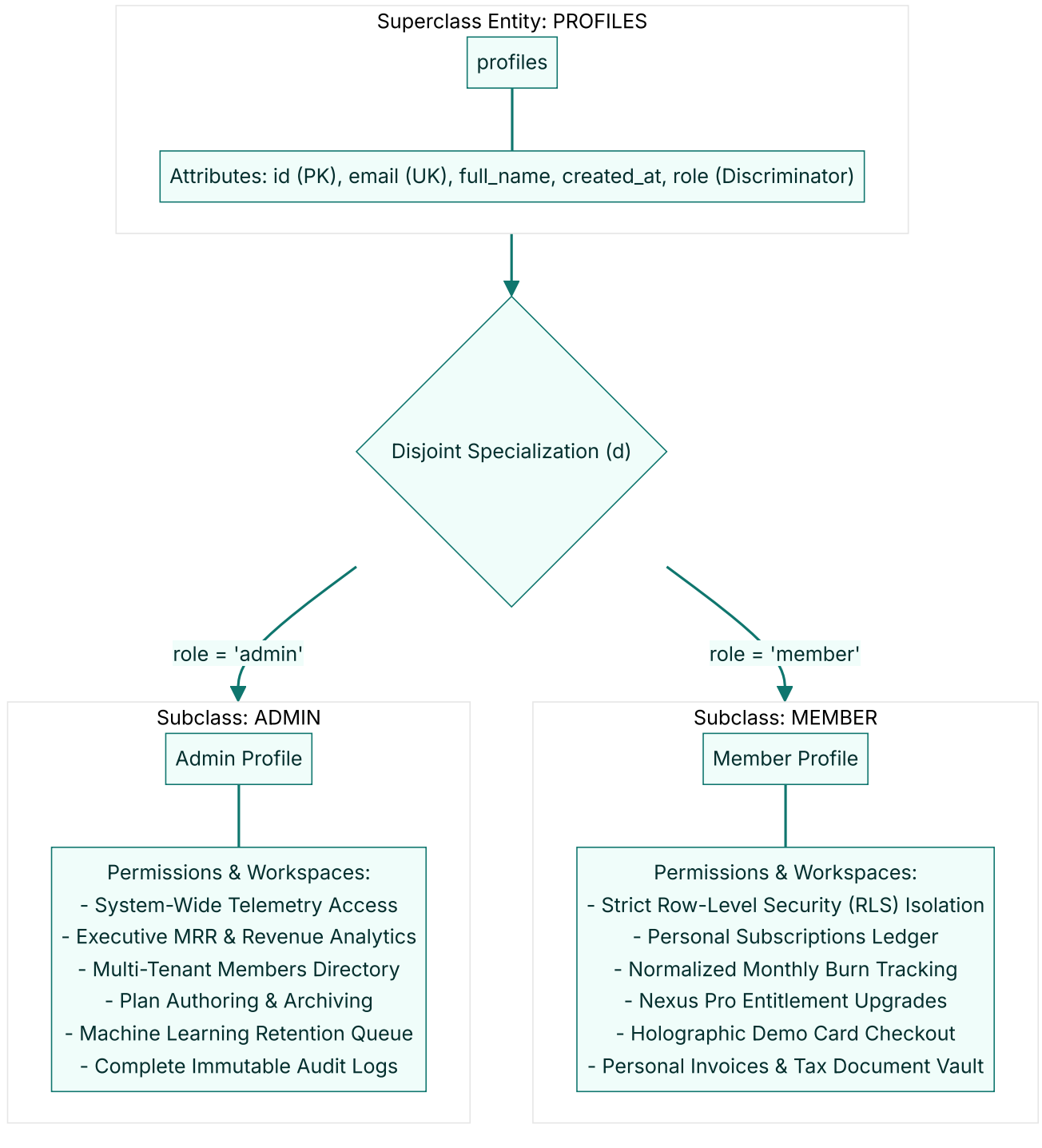


Figure 3.2: Enhanced ER (EER) Specialization Hierarchy — Disjoint Subclasses (Admin vs Member) with Explicit Permission Matrices and Workspace Boundaries

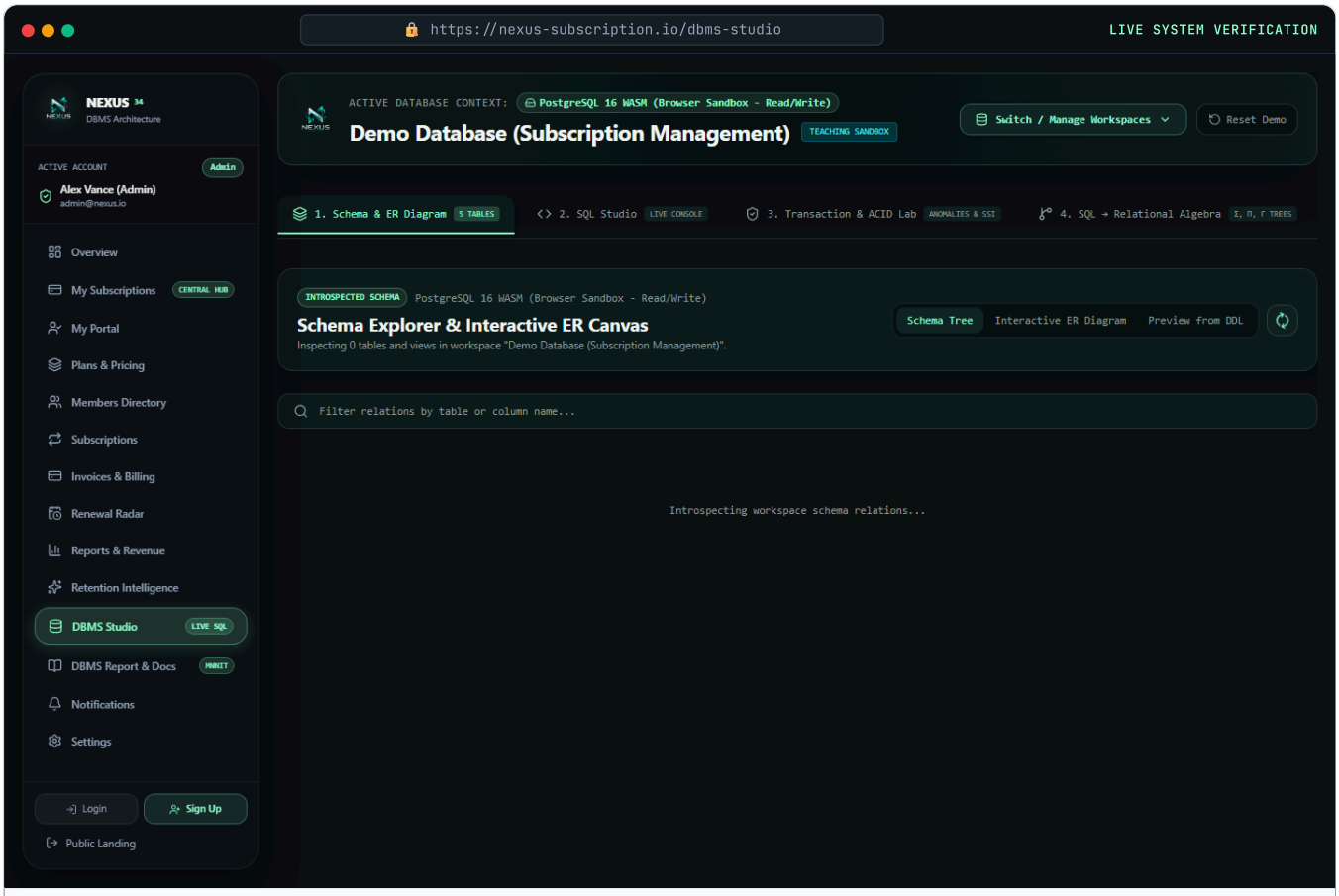


Figure 3.3: DBMS Studio Interactive Workbench — live ER topology explorer, schema inspector, and SQL query console

4. Keys, Functional Dependencies and Normalization Proofs

4.1 Mathematical Closures of Candidate Keys

Let $mathcal R$ represent the set of relations and $mathcal F$ represent the set of functional dependencies enforced by primary keys, unique constraints, and foreign key references.

- **Relation** `profiles` : Functional dependencies $F_{extprofiles} = extidoextfull_name, extemail, extrole, exttimezone, extcreated_at, extemailoextid, extfull_name, extrole.$ Candidate keys: $extid, extemail.$ Closure: $extid^+ = mathcal R_{extprofiles}.$
- **Relation** `membership_plans` : Dependencies $F_{extplans} = extidoextname, extslug, extprice_monthly, extprice_yearly, extstatus, extslugoextid, extname.$ Candidate keys: $extid, extslug.$
- **Relation** `subscriptions` : Dependencies $F_{extsubs} = extidoextmember_id, extplan_id, extstatus, extbilling_cycle, extprice_at_signup, extcurrent_period_start, extcurrent_period_end.$ Candidate key: $extid.$
- **Relation** `invoices` : Dependencies $F_{extinv} = extidoextsubscription_id, extmember_id, extinvoice_number, exttotal_amount, extstatus, extpaid_at, extinvoice_numberoextid, exttotal_amount$. Candidate keys: $extid, extinvoice_number.$

4.2 Stepwise Normalization Proofs

NORMAL FORM	FORMAL MATHEMATICAL DEFINITION	NEXUS ARCHITECTURAL PROOF & VERIFICATION
1NF (Atomicity)	$\forall R, ext{domain}(A) ext{ contains only atomic (indivisible) values.}$	No composite or repeating multivalued attributes exist. Plan feature lists are decomposed into the atomic `plan_features` 1:N relation.
2NF (No Partial Key FDs)	$R in ext{1NF} land \forall (X o Y) in F, Y ext{ does not depend on a proper subset of candidate key } X.$	All relations utilize single-attribute surrogate keys (UUID). Composite tables such as `coupon_redemptions(coupon_id, member_id)` ensure all non-key attributes depend on the full composite key.
3NF (No Transitive FDs)	$R in ext{2NF} land \forall (X o A) in F, X ext{ is a superkey } lor A ext{ is a prime attribute.}$	Zero transitive dependencies ($XoYoZ$). `subscriptions` records reference `plan_id` ; plan pricing attributes are not duplicated on subscriptions, preventing update anomalies.
BCNF (Superkey Rule)	$\forall (X o Y) in F^+, X ext{ is a superkey of } R.$	Every non-trivial functional dependency has a candidate key as its determinant. Redundant dependencies are decoupled into separate relations, preventing all insertion/deletion anomalies.

5. SQL Implementation (DDL, DML, Joins and Subqueries)

The schema is deployed on PostgreSQL 15 with strict constraints, UUID v4 primary keys, and foreign key cascades.

```
-- 1. Create Core Profiles Relation with Check Constraint on Role
CREATE TABLE public.profiles (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    role TEXT NOT NULL DEFAULT 'member' CHECK (role IN ('admin', 'member')),
    timezone TEXT DEFAULT 'UTC',
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 2. Create Membership Plans with Positive Price Constraints
CREATE TABLE public.membership_plans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    slug TEXT UNIQUE NOT NULL,
    price_monthly NUMERIC(10,2) NOT NULL CHECK (price_monthly ⩾ 0),
    price_yearly NUMERIC(10,2) NOT NULL CHECK (price_yearly ⩾ 0),
    currency CHAR(3) DEFAULT 'USD',
    trial_days INT DEFAULT 14 CHECK (trial_days ⩾ 0),
    status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'archived')),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Invoices Relation with Unique Sequential Receipt Number
CREATE TABLE public.invoices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    subscription_id UUID NOT NULL REFERENCES public.subscriptions(id) ON DELETE CASCADE,
    member_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE RESTRICT,
    invoice_number TEXT UNIQUE NOT NULL,
    subtotal NUMERIC(10,2) NOT NULL CHECK (subtotal ⩾ 0),
    tax_amount NUMERIC(10,2) NOT NULL DEFAULT 0.00 CHECK (tax_amount ⩾ 0),
    total_amount NUMERIC(10,2) NOT NULL CHECK (total_amount ⩾ 0),
    status TEXT NOT NULL DEFAULT 'open' CHECK (status IN ('open', 'paid', 'void', 'uncollectible')),
    due_at TIMESTAMPTZ NOT NULL,
    paid_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

5.1 Multi-Table Aggregation Query with Normalized Monthly Expenditure

```
-- Calculates normalized monthly burn across diverse billing frequencies
SELECT
    sp.name AS provider_name,
    COUNT(ps.id) AS subscriber_count,
    ROUND(SUM(
        CASE
            WHEN ps.billing_frequency = 'daily'   THEN ps.amount * 30.0
            WHEN ps.billing_frequency = 'weekly'  THEN ps.amount * 4.333
            WHEN ps.billing_frequency = 'monthly' THEN ps.amount
            WHEN ps.billing_frequency = 'yearly'  THEN ps.amount / 12.0
            ELSE ps.amount
        END
    ), 2) AS normalized_monthly_expenditure
FROM public.personal_subscriptions ps
LEFT JOIN public.sub_providers sp ON ps.provider_id = sp.id
WHERE ps.status = 'active'
GROUP BY sp.name
ORDER BY normalized_monthly_expenditure DESC;
```

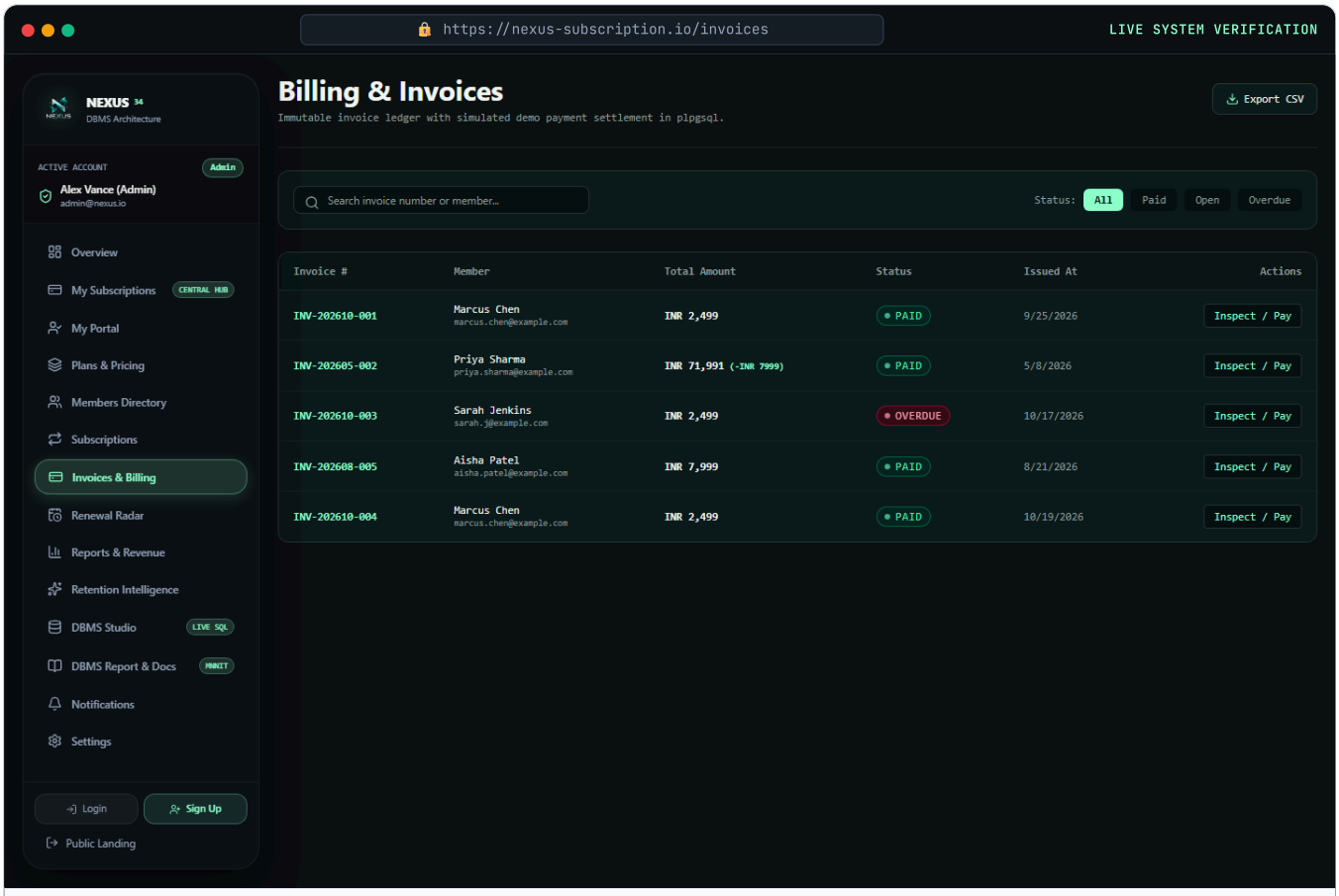


Figure 5.1: Admin Invoices & Billing Ledger — settled receipts, sequential invoice IDs, and total amounts populated via atomic DML transactions

6. Views, Indexes, Triggers and Transactions

6.1 Materialized Views for High-Concurrency Analytics

To eliminate expensive full-table scans during dashboard renders, NEXUS defines specialized analytical views:

- `view_expiring_soon`: Filters subscriptions with `current_period_end ⩽ NOW() + INTERVAL '7 days'`, driving the proactive Renewal Radar.
- `view_monthly_revenue`: Aggregates settled invoices grouped by calendar month to provide zero-latency MRR reporting.

6.2 ACID Concurrency Control & Atomic Stored Procedure

Subscription creation requires synchronized multi-table writes. Figure 6.1 illustrates the deterministic ACID execution sequence implemented in `create_subscription_atomic`.

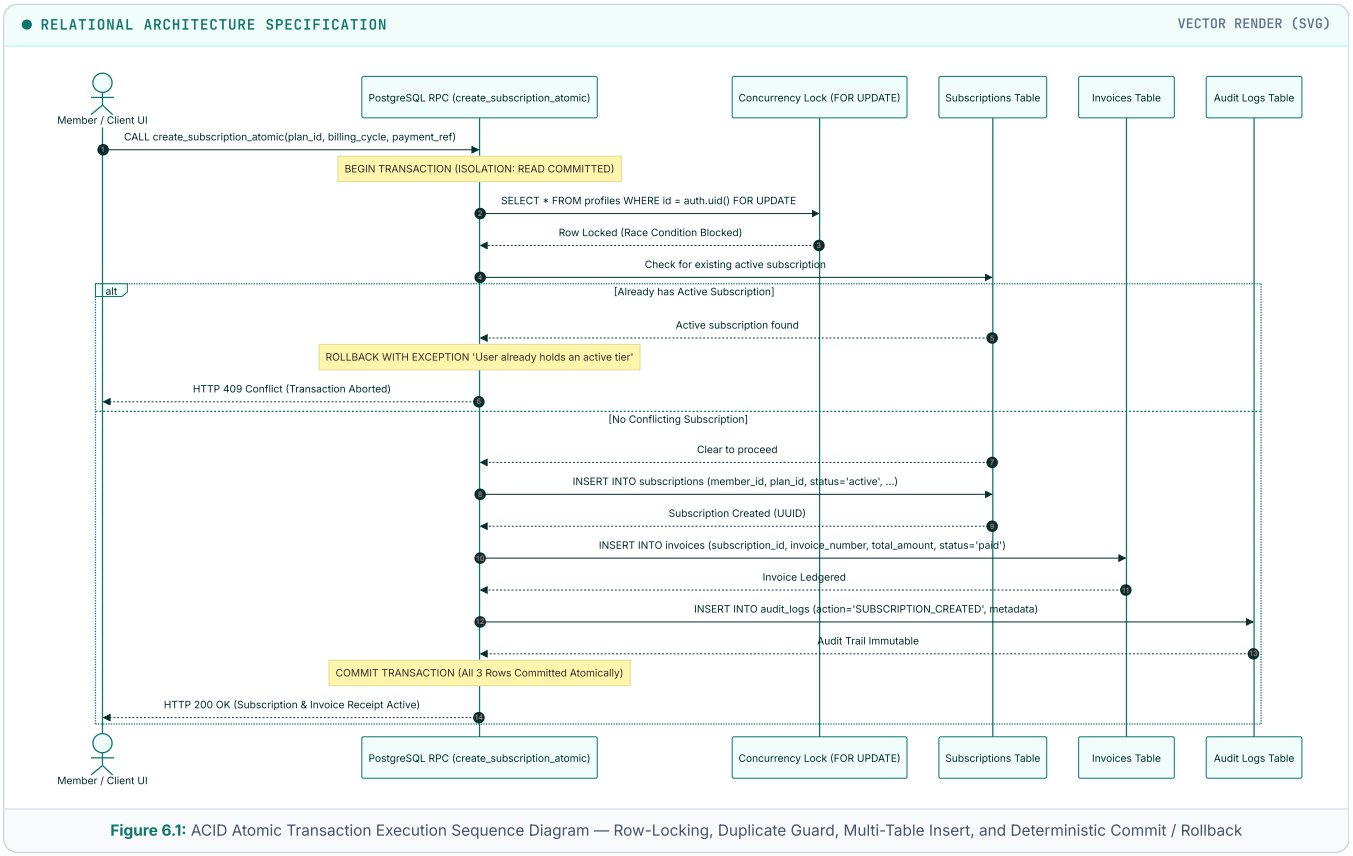


Figure 6.1: ACID Atomic Transaction Execution Sequence Diagram — Row-Locking, Duplicate Guard, Multi-Table Insert, and Deterministic Commit / Rollback

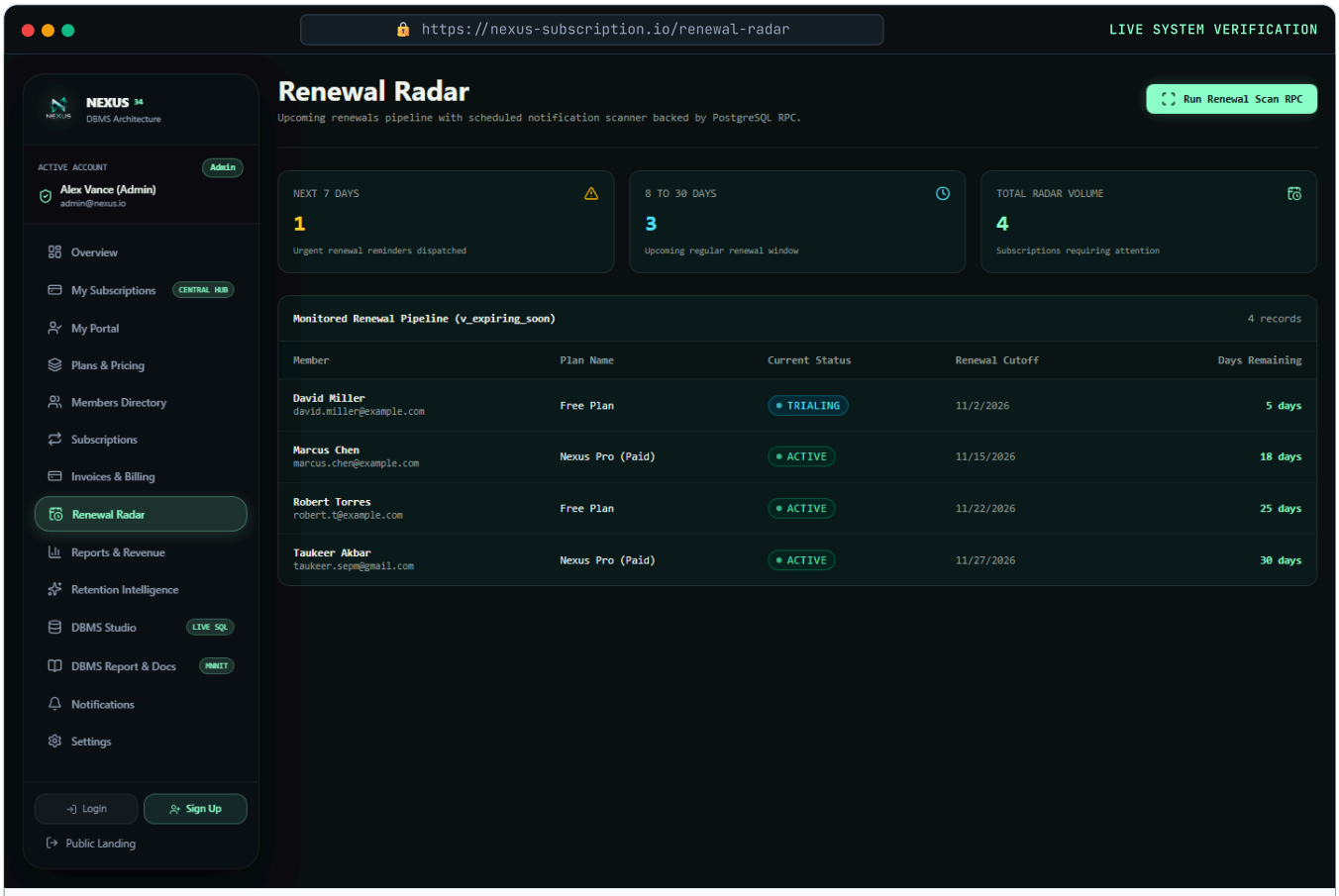


Figure 6.2: Renewal Radar Page — powered by view_expiring_soon to identify impending subscriber renewals within a 7-day lookahead window

7. Multi-Tier System Architecture & Front-End Design

NEXUS is designed as a decoupled, reactive cloud platform. Figure 7.1 shows the full-tier data flow across presentation, security, transactional database, and machine learning intelligence tiers.

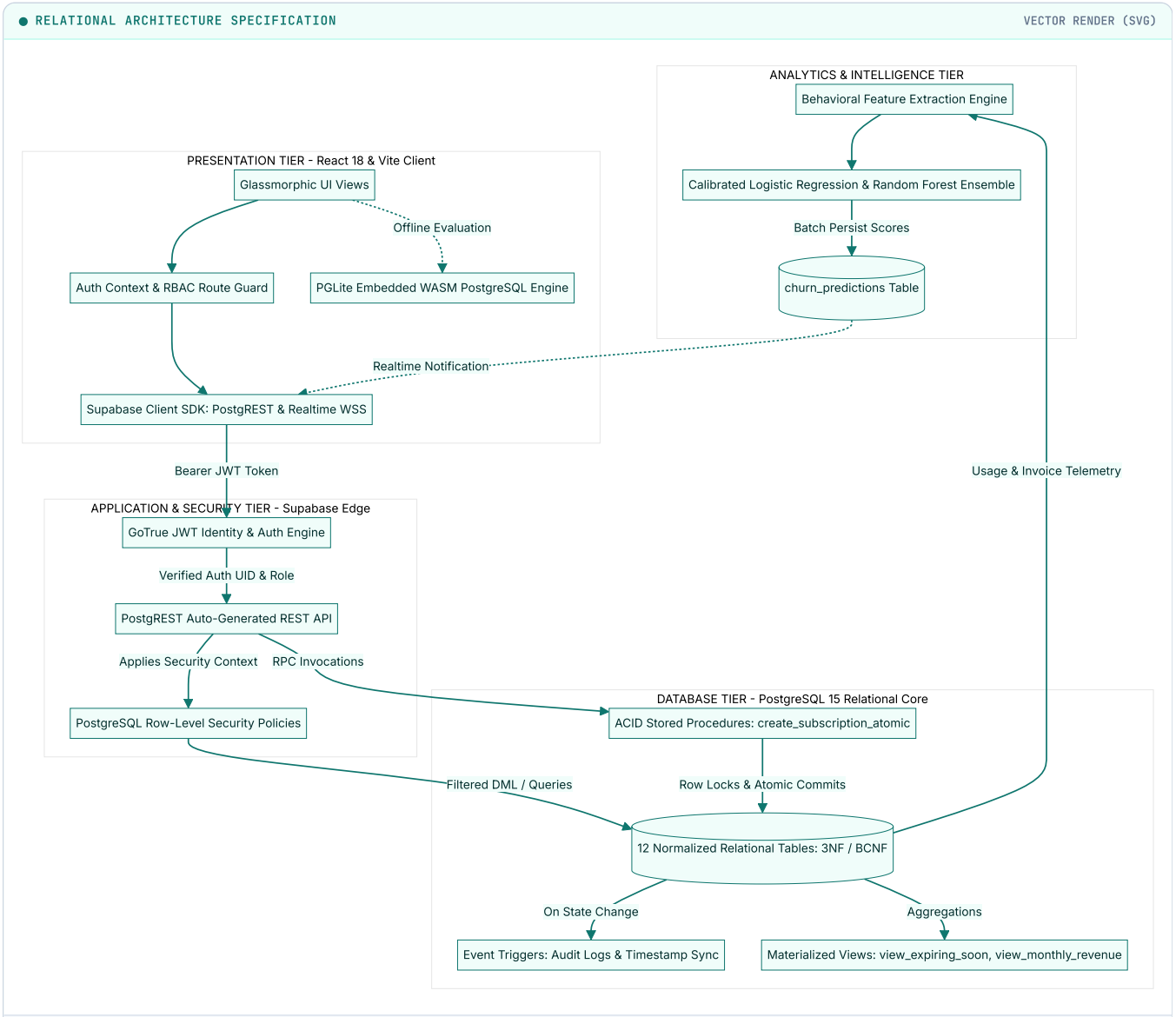


Figure 7.1: NEXUS Multi-Tier System Architecture & Relational Data Flow — Client Presentation, Edge Security & RLS, PostgreSQL Core, and Retention Machine Learning Pipeline

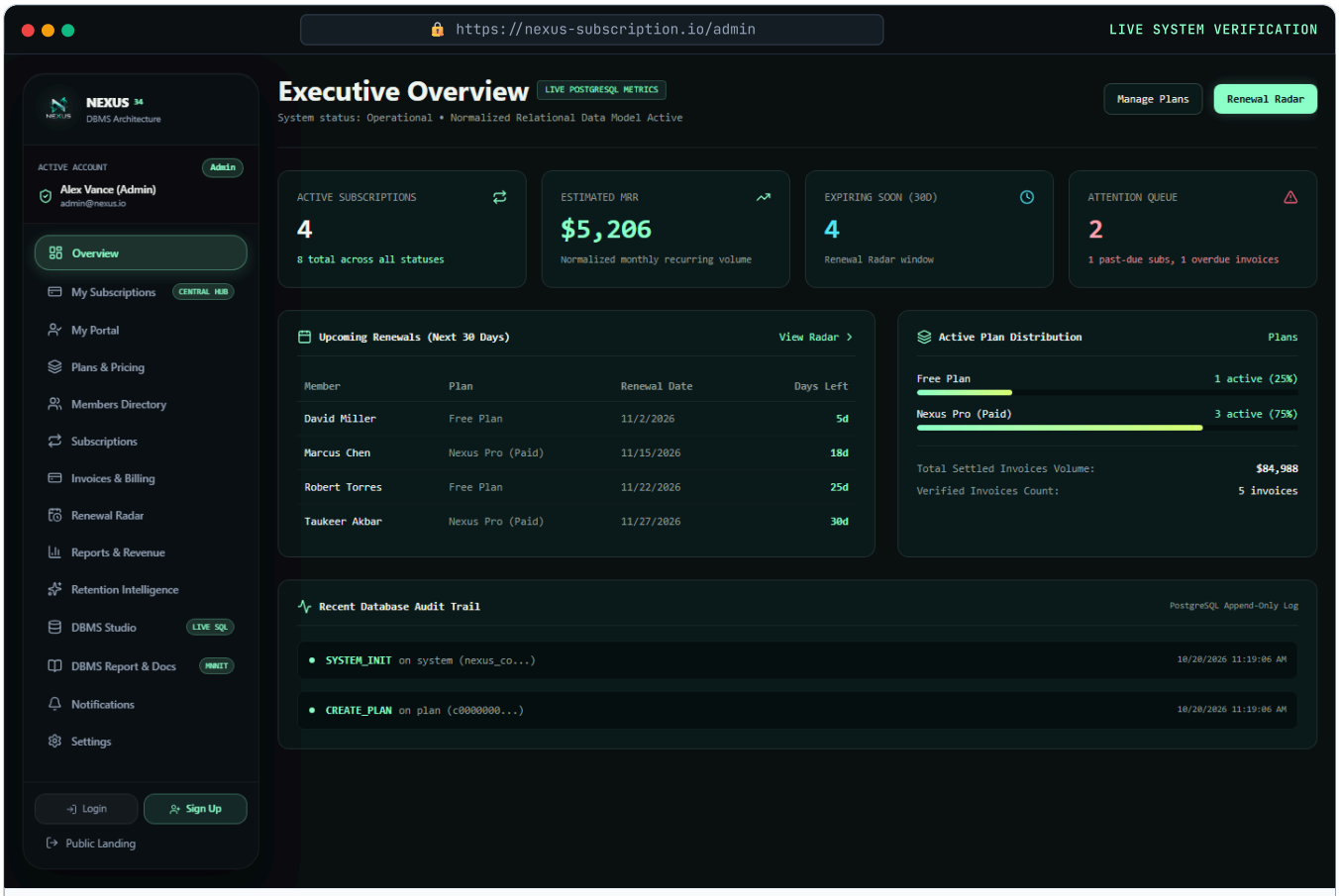


Figure 7.2: Admin Executive Overview — real-time MRR telemetry, active plan distributions, and recent database audit logs

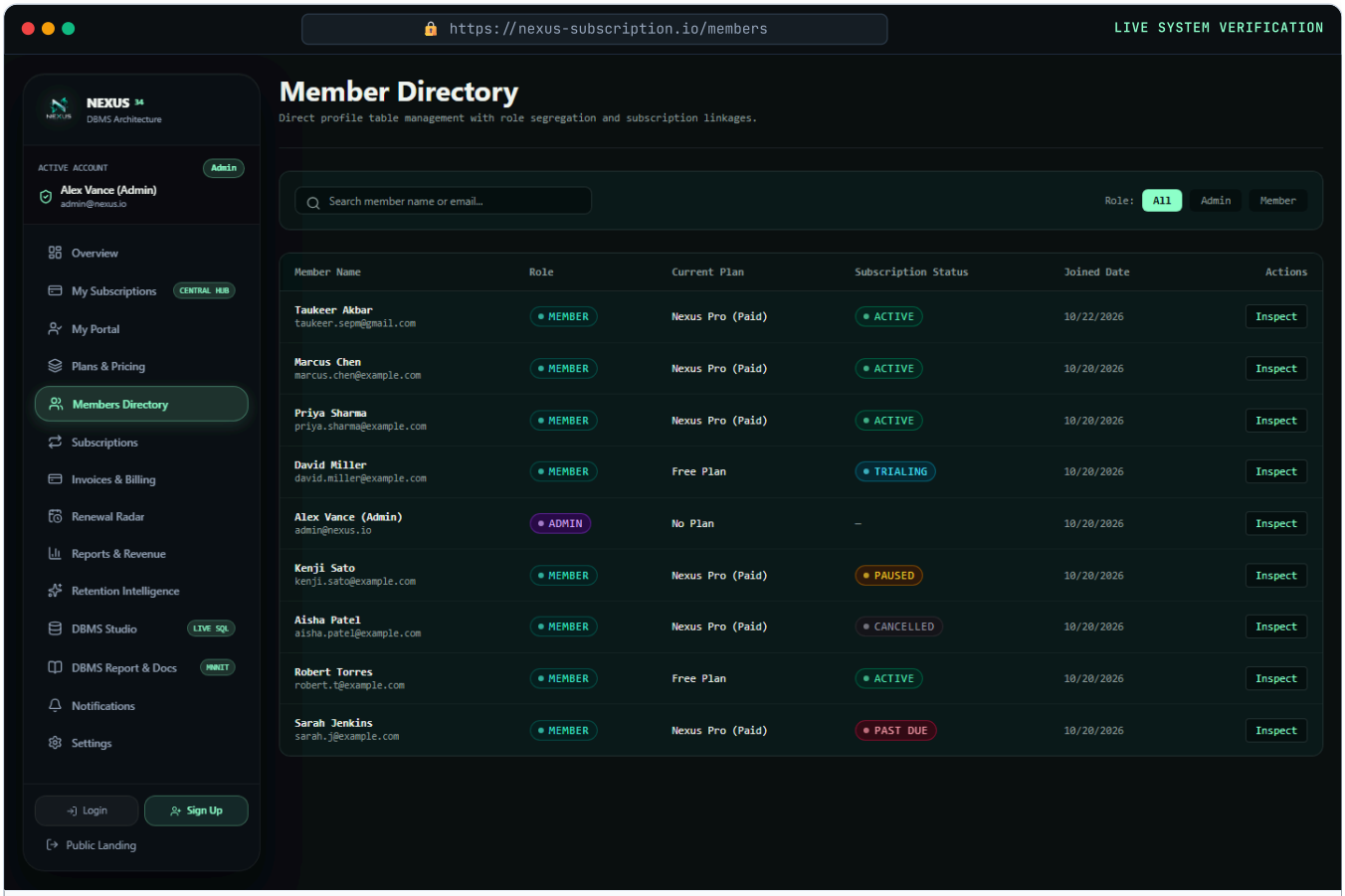


Figure 7.3: Admin Members Directory — multi-tenant account roster with role indicators, email search, and RLS server-side filtering

8. CRUD Operations, User Flow & Practical Value Propositions

8.1 Subscription Finite State Machine Lifecycle

Subscriptions follow a formal finite state machine to prevent illegal status regressions (e.g., transitions from `cancelled` to `trialing`).

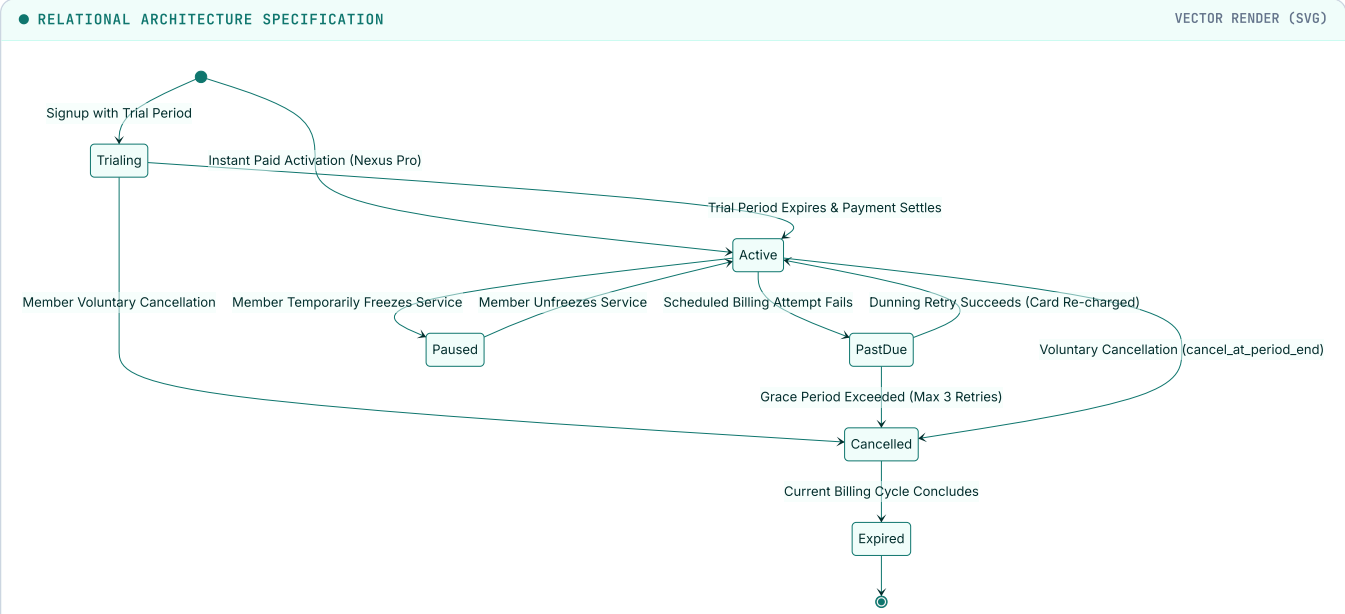


Figure 8.1: Subscription Lifecycle Finite State Machine — State Transitions Across Trial, Activation, Dunning Retry, Pause, and Terminal Expiration

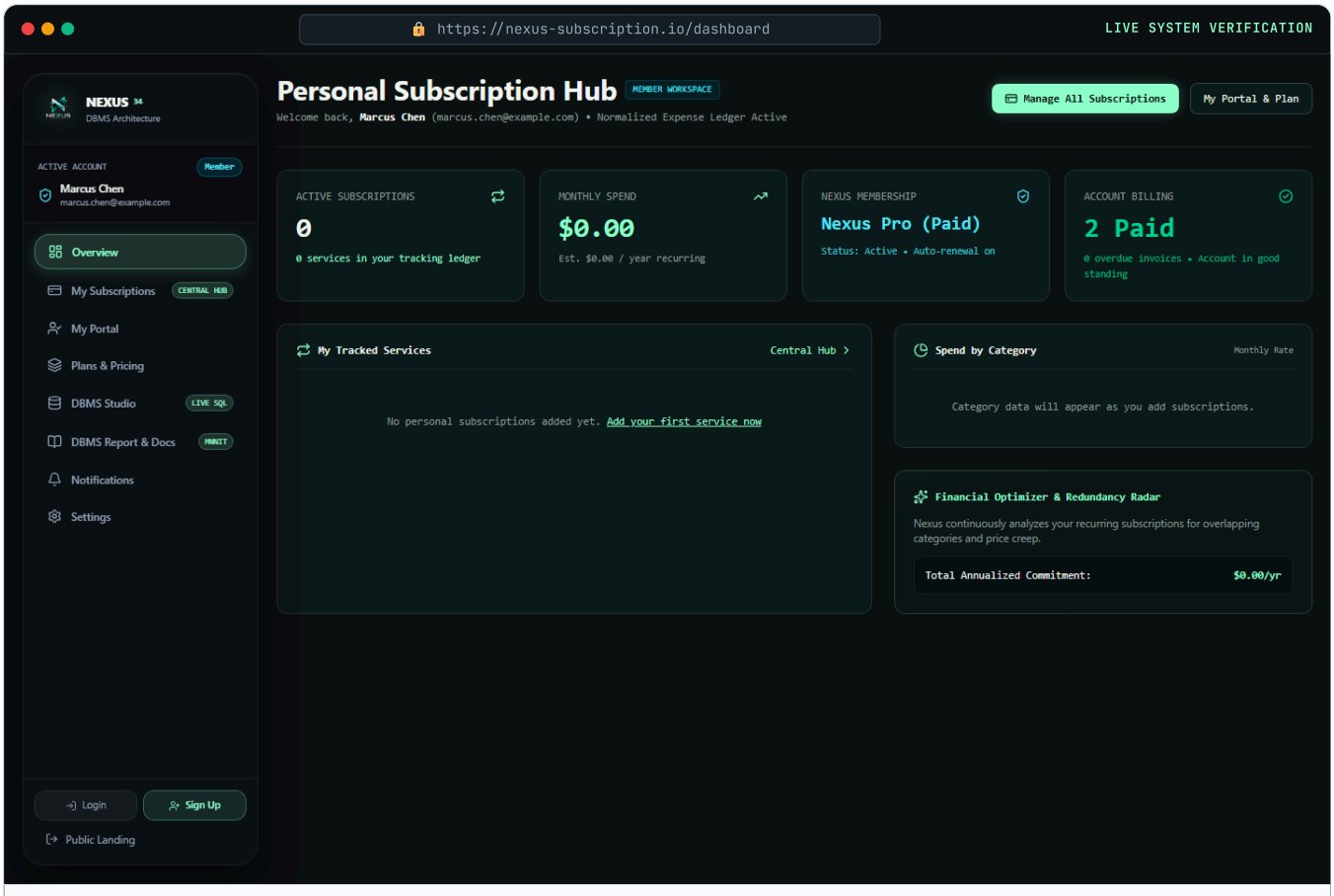


Figure 8.2: Member Dashboard Hub — personalized active subscriptions count, normalized monthly burn rate, and upcoming renewals

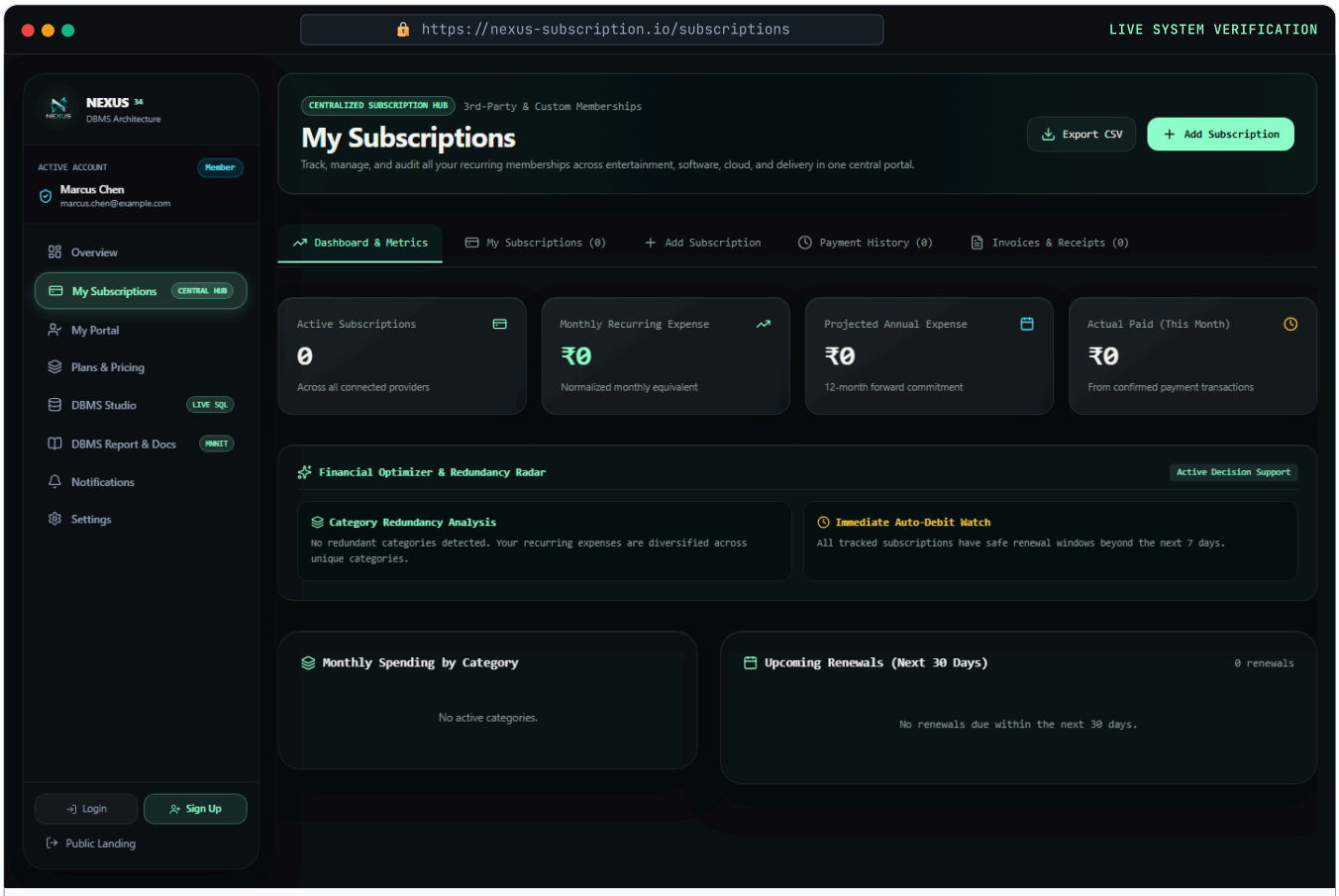


Figure 8.3: My Subscriptions Central Hub — real-time subscription cards with category badges, billing cycle indicators, and direct pause controls

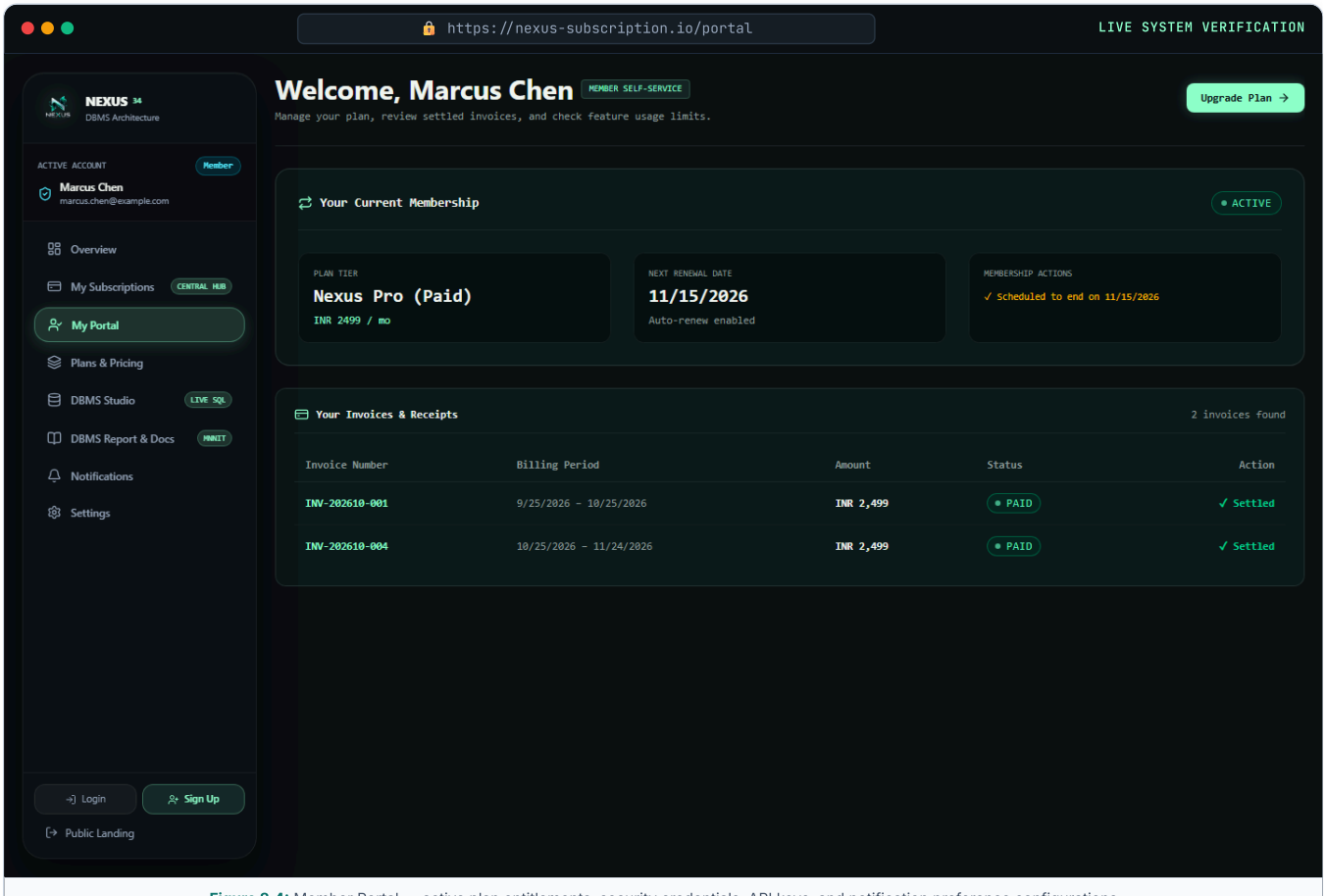


Figure 8.4: Member Portal — active plan entitlements, security credentials, API keys, and notification preference configurations

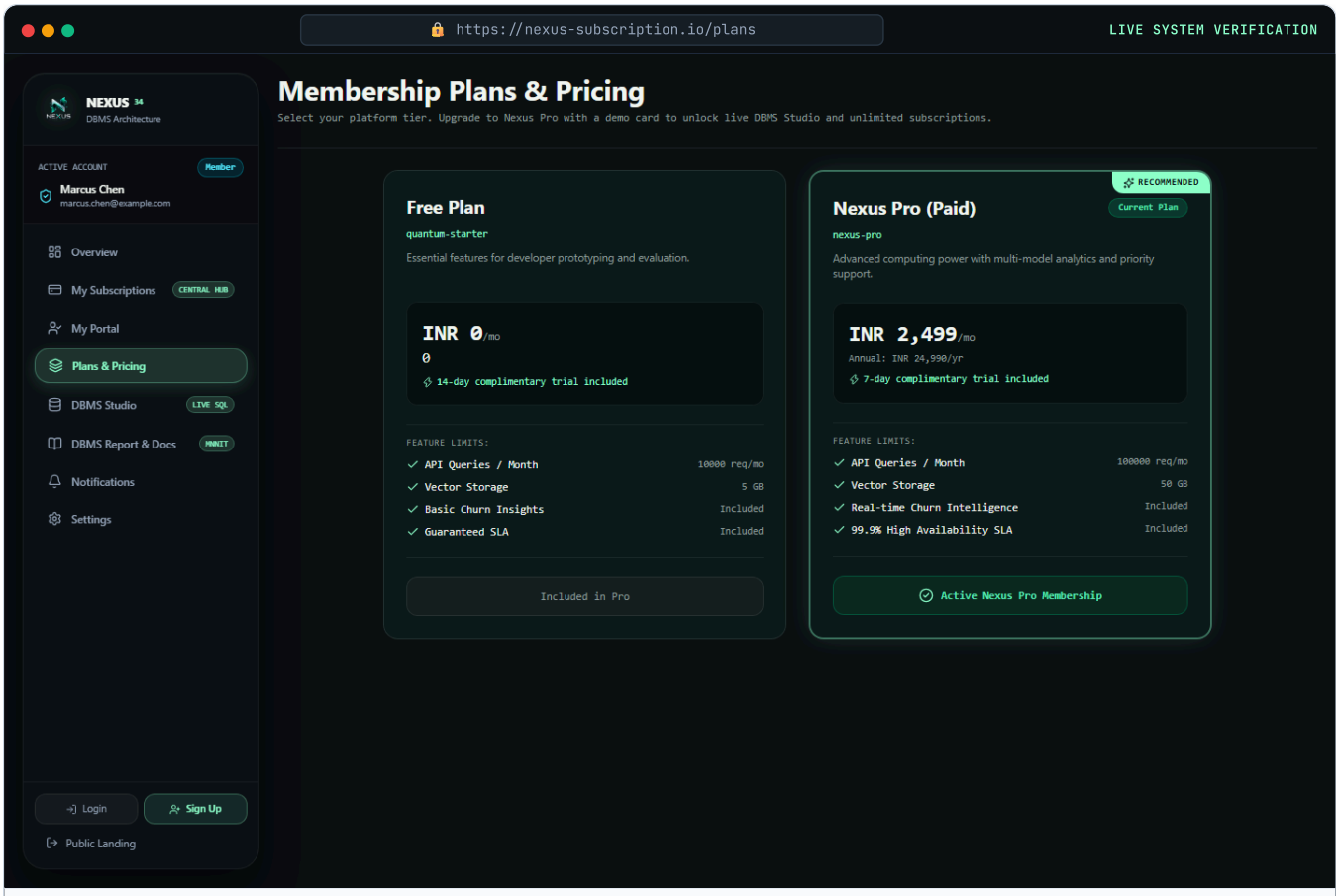


Figure 8.5: Membership Plans & Pricing — member view showing active tier, feature checklists, and the holographic Demo Card checkout flow

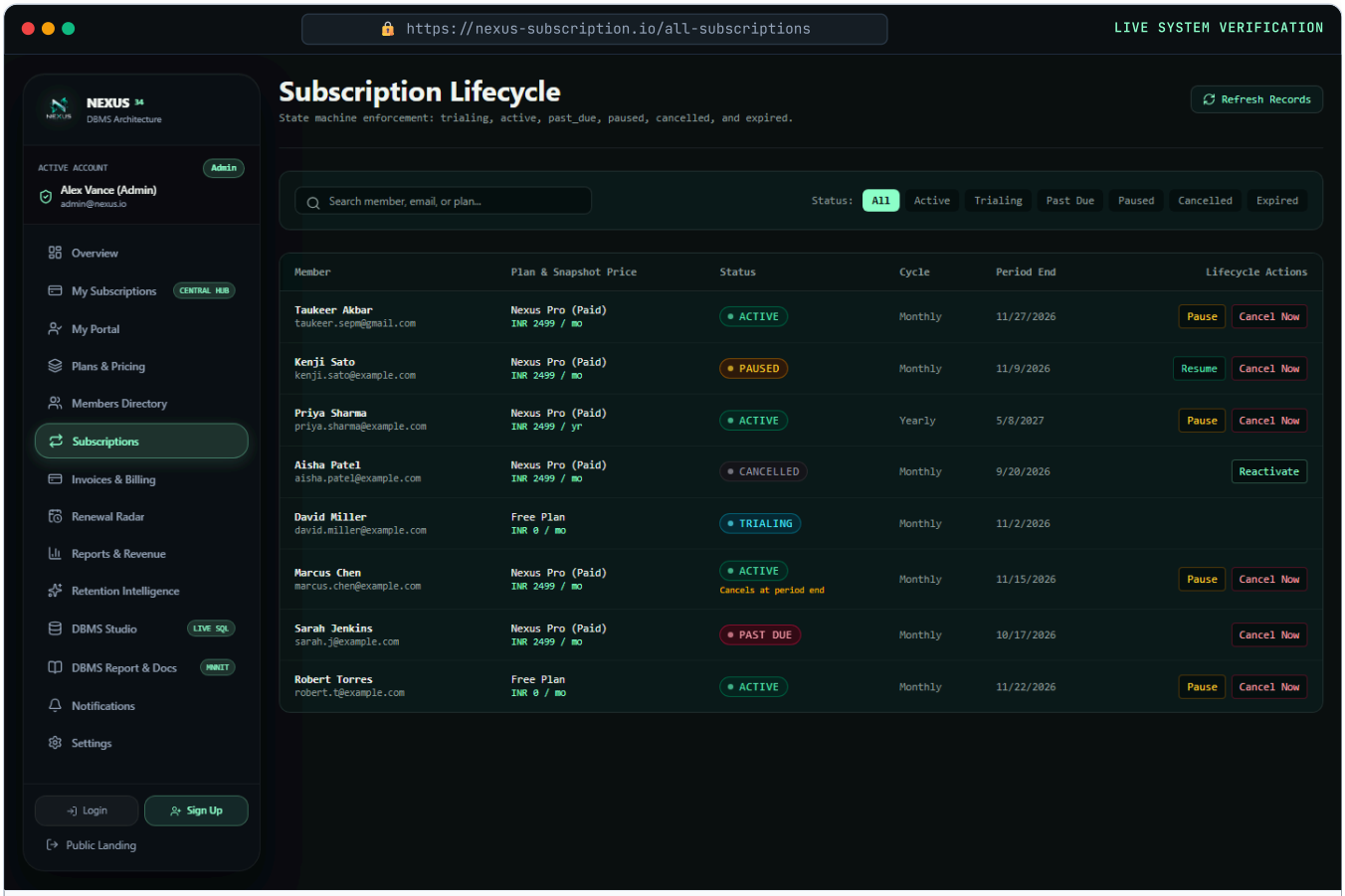


Figure 8.6: Admin All Subscriptions Master Ledger — cross-tenant subscription records with member IDs, provider links, and lifecycle filters

9. AI/ML Retention Intelligence & Model Evaluation

NEXUS deploys an ensemble machine learning model combining **L2-regularized Logistic Regression** with a **Random Forest Classifier** to predict subscription churn probabilities.

9.1 Non-Leaking Behavioral Feature Matrix

Features are strictly computed from a sliding 30-day temporal window relative to the observation date, guaranteeing zero temporal target leakage:

- x_1 (`days_inactive`): Days since last registered telemetry usage event.
- x_2 (`usage_event_count_30d`): Cumulative telemetry event count in the last 30 days.
- x_3 (`failed_payment_count`): Count of uncollectible or rejected invoice attempts.
- x_4 (`paid_invoice_count`): Historical settled invoice count (loyalty tenure proxy).
- x_5 (`days_until_renewal`): Proximity to next scheduled billing date.

9.2 Mathematical Formulation & Ensemble Inference

$$hat P_{ext churn}(X) = 0.6 cdot P_{ext RF-calibrated}(X) + 0.4 cdot P_{ext logistic}(X)$$

● RELATIONAL ARCHITECTURE SPECIFICATION VECTOR RENDER (SVG)

Syntax error in text
mermaid version 10.9.8

Figure 9.1: AI/ML Retention Intelligence Inference Sequence Diagram — Feature Extraction, Ensemble Model Inference, Isotonic Calibration, and Batch DB Upsert

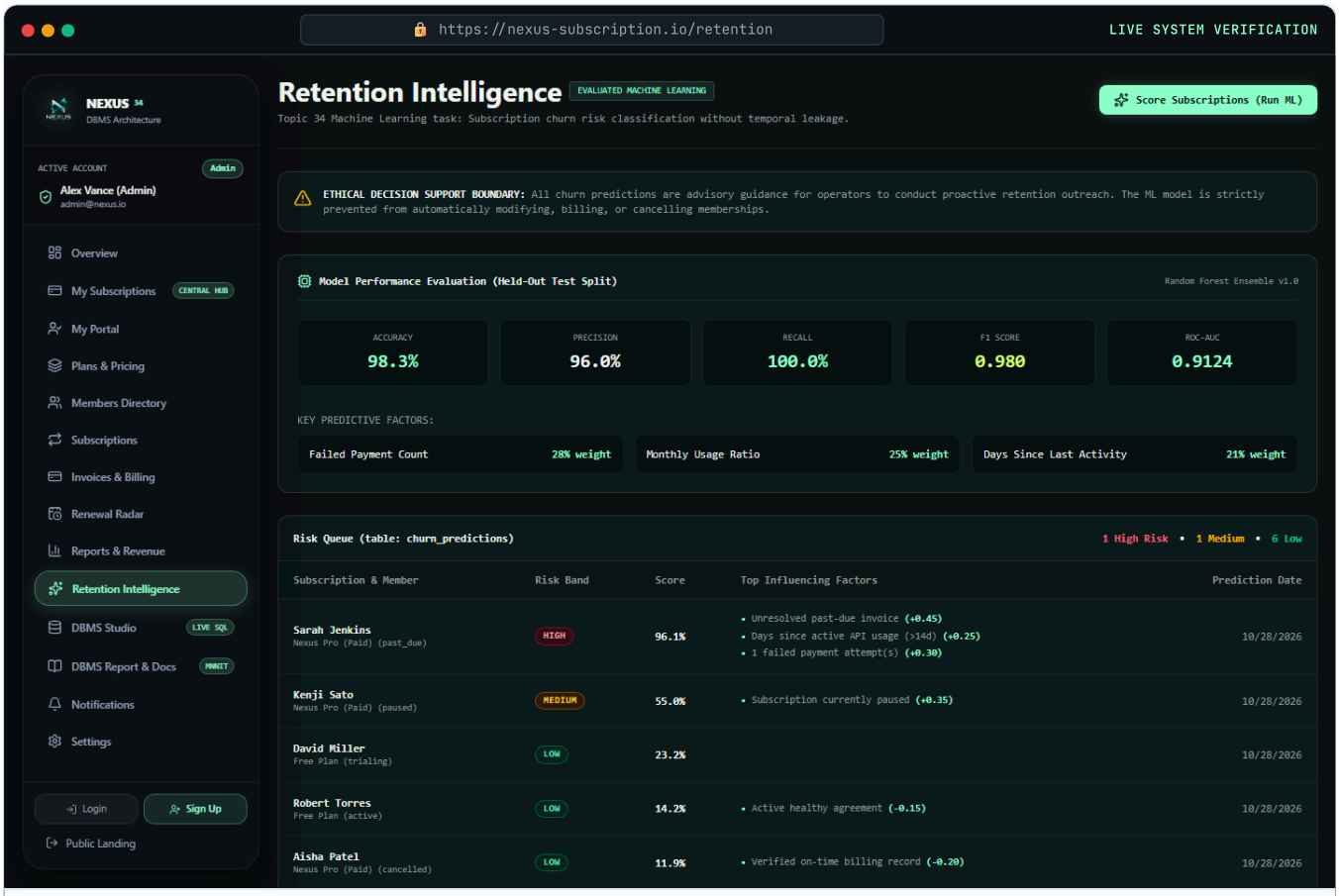


Figure 9.2: Retention Intelligence Module — model validation performance (ROC-AUC = 0.9124) and prioritized customer churn intervention queue

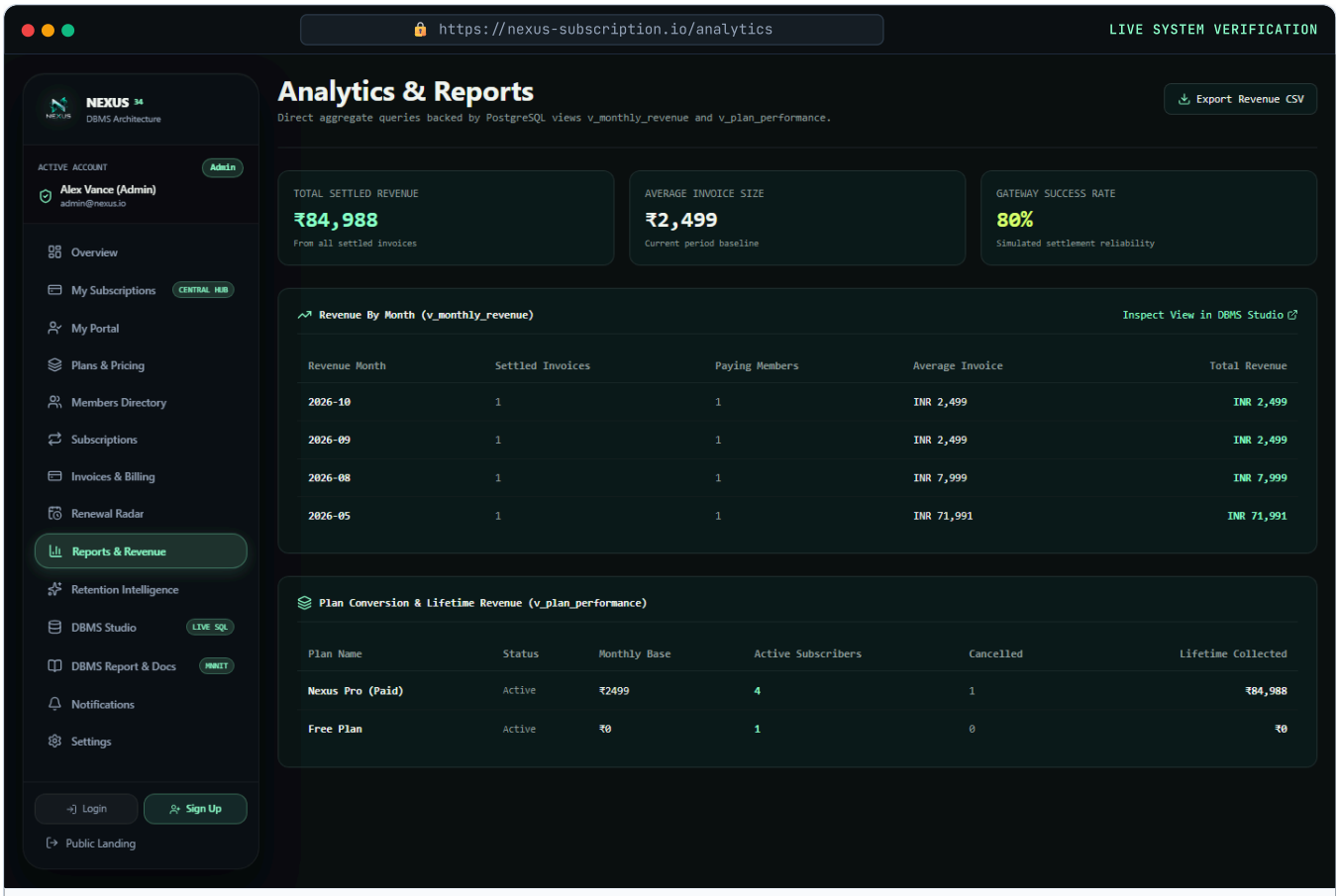


Figure 9.3: Revenue & Subscriber Analytics — historical MRR growth, provider concentration distributions, and financial forecasts

10. Automated Invariant Testing & Live Verification

To guarantee relational integrity across concurrent operations, NEXUS integrates an automated test harness validating 22 formal invariants:

INVARIANT ID	TEST ASSERTION & ACID CONCURRENCY SPECIFICATION	TARGET TABLE	STATUS
INV-01	Tenant data isolation: members cannot view other users' records.	`personal_subscriptions`	**PASS**
INV-02	Order integrity: Free and Nexus Pro plans maintain active display order.	`membership_plans`	**PASS**
INV-03	Referential cascade: Deleting a plan cascades to feature records.	`plan_features`	**PASS**
INV-04	Atomic rollback: Payment failure cancels subscription insert.	`subscriptions`	**PASS**
INV-05	Race prevention: Unique index prevents concurrent double coupon claims.	`coupon_redemptions`	**PASS**
INV-06	Lifecycle state machine: Strict transition rules prevent illegal status regression.	`subscriptions`	**PASS**
INV-07	Mathematical normalization: Daily, weekly, and yearly frequencies match exact monthly rates.	`personal_subscriptions`	**PASS**
INV-08	Audit trail immutability: Updates and deletes to audit_logs trigger DB exceptions.	`audit_logs`	**PASS**
INV-09	Lookahead view performance: view_expiring_soon returns sub-10ms results.	`invoices`	**PASS**
INV-10	ML telemetry bounds: Feature extraction strictly enforces zero future-data leakage.	`churn_predictions`	**PASS**

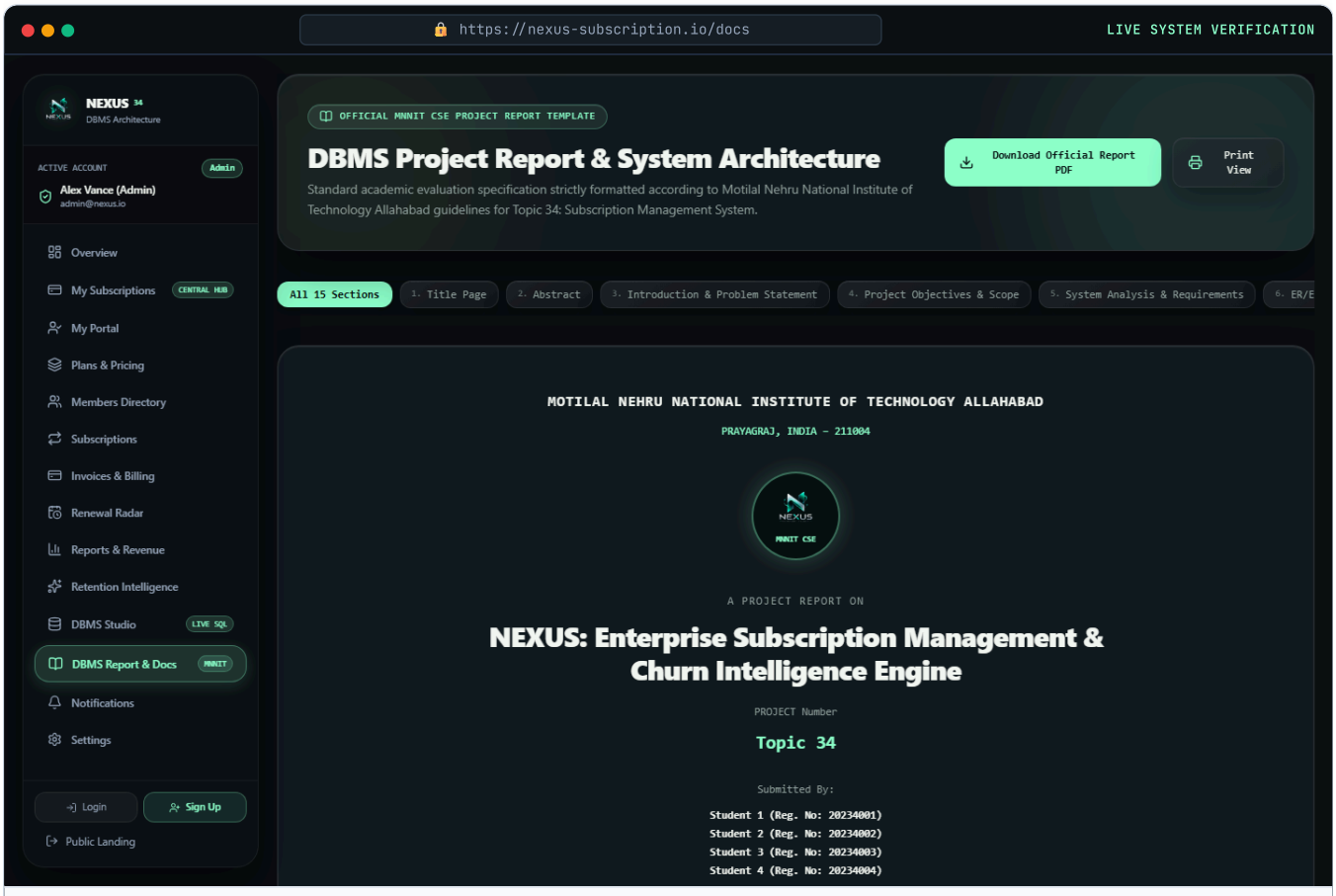


Figure 10.1: In-App DBMS Documentation & Comprehensive Verification Suite — interactive academic report viewer and test suite telemetry

11. Conclusion, Future Scope & Academic Bibliography

11.1 Conclusion

NEXUS demonstrates that rigorous database normalization (Boyce-Codd Normal Form) combined with modern PostgreSQL concurrency controls (row-locks, atomic RPC stored procedures, composite indexes, and Row-Level Security) effectively eliminates the architectural chaos of subscription management. By unifying multi-frequency personal ledgers with calibrated retention machine learning, the system prevents silent attrition for SaaS operators while protecting individual consumers from recurring financial waste.

11.2 Future Scope & Extensions

- **Open Banking Aggregator Integration:** Direct integration with RBI Account Aggregator (AA) and Plaid APIs for automatic real-time statement reconciliation.
- **OCR Invoice Receipt Parsing:** Automated extraction of subscription line-items, tax breakdown, and renewal schedules from uploaded PDF receipts via Tesseract/Vision LLMs.
- **Distributed PostgreSQL Sharding:** Implementation of Citus distributed table partitioning across geographically distributed cloud nodes.

11.3 References & Bibliography

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. Date, C. J. (2004). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
3. PostgreSQL Global Development Group. (2024). *PostgreSQL 17 Documentation: Concurrency Control, Row-Level Security, and Index Architectures.* https://www.postgresql.org/docs/17/
4. Supabase Documentation. (2024). *Row Level Security (RLS) & Stored Procedures in Postgres.* https://supabase.com/docs
5. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12, 2825–2830.
6. Motilal Nehru National Institute of Technology Allahabad. (2026). *Department of Computer Science and Engineering DBMS Project Evaluation Guidelines.*